

# REST API 기초

# 학습 체크리스트 (1/2)

---

- ☐ SSR과 REST API의 책임 분담 차이와 REST 제약 조건 이해
- ☐ 리소스 중심 URI 설계와 HTTP 메서드·상태 코드 선택 기준 파악
- ☐ `springdoc-openapi`로 Swagger UI를 미리 준비하는 방법 습득
- ☐ 직렬화·역직렬화가 `@RestController` 응답에서 일어나는 지점 이해

# 학습 체크리스트 (2/2)

---

☐ 요청·응답 DTO를 `record`로 분리해야 하는 이유 파악

☐ `@Valid`로 요청 본문 검증을 거는 방법과 REST에서의 처리 방식 습득

☐ `ResponseEntity`로 상태 코드·헤더·본문을 제어하는 방법 습득

☐ 게시판 CRUD API 구현과 페이징 응답 계약 고정 방식 파악

# 학습 범위

---

- 선행 조건: SSR 게시판 CRUD와 작성자 권한 검증까지 완성
- 1. REST 설계: 리소스와 HTTP 의미론 중심의 API 설계 원칙
- 2. 문서화 준비: `springdoc-openapi` 와 Swagger UI 세팅
- 3. 컨트롤러 구현: `@RestController`, DTO, 검증, `ResponseEntity`
- 4. 게시판 API: 게시글 CRUD와 페이징 응답, 명세 보강

# 전체 진행 순서

---



# REST와 HTTP 설계

# SSR에서 REST로: 서버 책임의 이동

---

- 기존 SSR: 컨트롤러가 `Model` 을 채우고 Thymeleaf가 완성된 HTML을 렌더링
- 이번 차시부터: 컨트롤러는 뷰 이름 대신 데이터(JSON)만 반환
- 화면 책임 이동: 화면을 그리는 일은 클라이언트(웹, 모바일, 다른 서버)의 몫
- 본질: 반환 타입 변경이 아니라 서버·클라이언트 책임 재분배 설계 문제
- 환경: Spring Boot 4.1(Spring Framework 7) 기준, `spring-boot-starter-webmvc` 사용

# SSR과 REST API 비교

---

| 구분      | SSR (Thymeleaf) | REST API         |
|---------|-----------------|------------------|
| 응답 본문   | 완성된 HTML        | JSON 데이터         |
| 요청 간 상태 | 세션 활용이 흔함       | 각 요청이 문맥 포함(무상태) |
| 화면 책임   | 서버가 화면을 완성      | 클라이언트가 화면을 구성    |
| 재사용성    | 이 화면 전용         | 웹·모바일·서버가 함께 사용  |

- 같은 `GET /boards` 도 SSR은 HTML 문서를, REST는 목록 JSON을 반환



# REST란 무엇인가

---

## REST (Representational State Transfer)

로이 필딩의 박사 논문에서 제시된, 자원을 표현(representation) 형태로 주고받는 분산 시스템 설계 스타일

- 제약을 모두 만족하면 RESTful하다고 부름
- REST는 프로토콜이 아니라 아키텍처 스타일

# REST의 제약 조건

---

| 제약 조건               | 의미                                 |
|---------------------|------------------------------------|
| 클라이언트-서버            | 화면과 데이터 저장을 분리해 독립적으로 발전           |
| 무상태(Stateless)      | 요청 간 클라이언트 상태를 보관하지 않음             |
| 캐시 가능               | 응답의 캐시 가능 여부를 명시                   |
| 계층화                 | 프록시·게이트웨이 존재를 클라이언트가 몰라도 됨         |
| 균일한 인터페이스           | 리소스 식별, 표현 조작, 자기 서술적 메시지, HATEOAS |
| (선택) Code-on-Demand | 서버가 실행 코드를 내려 클라이언트 기능 확장          |

# 무상태(Stateless)가 뜻하는 것

---

- 의미: 이전 요청의 대화 문맥에 의존하지 않고 각 요청만으로 처리 가능해야 함
- 오해 정정: 서버가 DB·캐시·세션을 안 쓴다는 뜻이 아님
- 인증 반영: 매 요청에 쿠키나 `Authorization` 헤더가 함께 전달돼야 함
- JWT는 선택지: REST의 필수 조건이 아니며, `HttpOnly` 세션 쿠키 방식도 널리 쓰임

# CSR이 화면을 그리는 순서

---



# API 통신 아키텍처 비교

---

|       |           |              |                  |          |
|-------|-----------|--------------|------------------|----------|
| 구분    | REST      | SOAP         | gRPC             | GraphQL  |
| 전송 방식 | HTTP      | HTTP, SMTP 등 | HTTP/2           | HTTP     |
| 직렬화   | JSON, XML | XML(Strict)  | Protocol Buffers | JSON     |
| 모델    | 리소스 중심    | 메시지 중심       | RPC & 스트리밍       | 쿼리 언어    |
| 유스케이스 | 범용 웹/모바일  | 금융·공공 Legacy | 마이크로서비스 간        | 다양한 디바이스 |

- tRPC는 TypeScript 모노레포에서 코드 생성 없는 타입 안전성을 제공하는 별도 선택지

# JSON이 사실상 표준이 된 이유

---

- JS 생태계 결합: `JSON.parse()`, `JSON.stringify()` 내장
- 간결한 텍스트 표현: XML 대비 태그·엔벨롭 오버헤드가 적음
- 가독성: Key:Value, 배열, 숫자, 문자열, 불리언, null만으로 구성
- 언어 독립 표준(RFC 8259): 거의 모든 언어가 파서를 기본 탑재

# 흔한 URI 설계 실수

| 나쁜 예                                | 좋은 예                          | 이유               |
|-------------------------------------|-------------------------------|------------------|
| <code>GET /getBoards</code>         | <code>GET /boards</code>      | 동사 대신 메서드로 표현    |
| <code>POST /boards/create</code>    | <code>POST /boards</code>     | 컬렉션에 POST면 충분    |
| <code>GET /boards/1/delete</code>   | <code>DELETE /boards/1</code> | 삭제는 DELETE 메서드로  |
| <code>POST /updateBoard?id=1</code> | <code>PUT /boards/1</code>    | 리소스는 경로, 동작은 메서드 |
| <code>GET /Boards/1</code>          | <code>GET /boards/1</code>    | 소문자 규칙 준수        |

- 표기 규칙: 복수 명사·소문자·하이픈을 쓰고 확장자와 끝 slash는 붙이지 않음
- 계층 표현: 하위 리소스는 `/boards/{id}/comments` 처럼 경로로 표현

# 경로 변수와 쿼리 스트링의 역할 분담

---

|       |                                         |                                       |
|-------|-----------------------------------------|---------------------------------------|
| 구분    | URI Path ( <code>@PathVariable</code> ) | 쿼리 스트링 ( <code>@RequestParam</code> ) |
| 역할    | 리소스 식별, 계층 관계 표현                        | 필터·정렬·페이징·검색 조건                       |
| 필수 여부 | 경로 템플릿에서 필수                             | 계약에 따라 필수/선택                          |
| 핵심 질문 | "어떤 리소스인가?"                             | "어떻게 가져올 것인가?"                        |



# HTTP 메서드의 안전성과 멍등성

| 메서드    | 안전 | 멍등     | 본문    | 게시판 예시                |
|--------|----|--------|-------|-----------------------|
| GET    | O  | O      | 없음    | GET /boards/1         |
| POST   | X  | X      | 있음    | POST /boards 생성       |
| PUT    | X  | O      | 있음    | PUT /boards/1 전체 교체   |
| PATCH  | X  | 보장 안 됨 | 있음    | PATCH /boards/1 부분 수정 |
| DELETE | X  | O      | 보통 없음 | DELETE /boards/1      |

- PUT은 리소스 전체를 교체하고, PATCH는 지정한 필드만 수정

# 성공을 알리는 상태 코드

---

| 코드             | 이름       | 언제       |
|----------------|----------|----------|
| 200 OK         | 성공       | 조회·수정 성공 |
| 201 Created    | 생성 성공    | 새 리소스 생성 |
| 204 No Content | 본문 없는 성공 | 삭제 성공    |

- 201 응답에는 `Location` 헤더에 새 리소스 URI(`/boards/{id}`)를 담음

# 실패를 알리는 상태 코드

---

| 코드                        | 이름     | 언제              |
|---------------------------|--------|-----------------|
| 400 Bad Request           | 잘못된 요청 | 형식 오류, 검증 실패    |
| 401 Unauthorized          | 인증 안 됨 | 인증 정보 없음/무효     |
| 403 Forbidden             | 권한 없음  | 인증됐으나 접근 권한 없음  |
| 404 Not Found             | 리소스 없음 | 해당 id 리소스 부재    |
| 409 Conflict              | 충돌     | 중복 등록, 동시 수정 충돌 |
| 500 Internal Server Error | 서버 오류  | 예상치 못한 내부 오류    |

- 401은 "누구인지 확인 안 됨", 403은 "확인됐지만 권한 없음"

# 목록 조회 요청과 응답

---

```
1 GET /boards?page=0&size=20&sort=createdAt,desc HTTP/1.1
2 Host: api.example.com
3 Accept: application/json
4
5 HTTP/1.1 200 OK
6 Content-Type: application/json
7
8 { "content": [{ "id": 2, "title": "공지사항" }], "page": 0, "size": 20 }
```

# API 문서화 환경 준비

# OpenAPI 명세와 Swagger UI의 역할

---

## OpenAPI 명세 (OpenAPI Specification)

API의 경로·요청·응답 구조를 정의하는 JSON/YAML 규격

- 명세는 규격: API 계약을 기계가 읽을 수 있는 형태로 표현
- Swagger UI는 뷰어이자 테스트 도구: 명세를 화면에 렌더링하고 요청까지 보낼 수 있게 함
- 자동 생성: springdoc이 컨트롤러·DTO를 분석해 기본 명세를 만들어 줌

# springdoc 의존성 추가

---

```
1 dependencies {  
2     implementation 'org.springdoc:springdoc-openapi-starter-webmvc-ui:3.0.3'  
3 }
```

# 기동하면 열리는 기본 경로

---

| 경로                            | 내용                   |
|-------------------------------|----------------------|
| <code>/v3/api-docs</code>     | OpenAPI 명세 데이터(JSON) |
| <code>/swagger-ui.html</code> | Swagger UI 진입점       |

- 운영 환경 주의: `prod` 프로파일에서는 노출을 끄거나 접근 제어로 보호
- 숨김은 대책이 아님: UI를 감추는 것이 보안 대책을 대신하지는 않음



# Swagger UI 기본 설정

---

```
1  springdoc:  
2    api-docs:  
3      path: /v3/api-docs  
4    swagger-ui:  
5      path: /swagger-ui.html  
6      operations-sorter: method  
7      tags-sorter: alpha  
8    default-produces-media-type: application/json
```

# API 전역 메타데이터 등록

---

```
1  @Configuration
2  class OpenApiConfig {
3      @Bean
4      OpenAPI boardApiOpenApi() {
5          return new OpenAPI().info(new Info()
6              .title("게시판 API").version("v1")
7              .description("게시글 CRUD REST API 명세"));
8      }
9  }
```

# REST 컨트롤러의 구조

# 직렬화와 역직렬화

## 직렬화 (Serialization)

메모리 상의 객체를 JSON 같은 전송 가능한 문자열 표현으로 바꾸는 과정

| 구분   | 변환        | Spring MVC 용도                       |
|------|-----------|-------------------------------------|
| 직렬화  | 객체 → JSON | 응답 반환( <code>@ResponseBody</code> ) |
| 역직렬화 | JSON → 객체 | 요청 처리( <code>@RequestBody</code> )  |

# JSON 본문이 객체로 오가는 경로

---



# @RestController가 대신 해주는 일

---

- 합성 애너테이션: `@Controller` + `@ResponseBody` 를 합쳐 놓은 것
- 반환값이 곧 본문: 메서드 반환값 자체가 응답 본문이 됨
- 뷰 리졸버 생략: 반환값이 뷰 리졸버를 거치지 않고 바로 변환됨
- Boot 4 기본 컨버터: Jackson 3 기반 `JacksonJsonHttpMessageConverter` 사용
- 문자열 반환 주의: 뷰 이름이 아니라 순수 텍스트 본문 그대로 나감

# 요청 매핑 어노테이션

---

| 어노테이션                        | HTTP 메서드                     | 용도        |
|------------------------------|------------------------------|-----------|
| <code>@RequestMapping</code> | 전체 또는 <code>method</code> 지정 | 클래스 공통 경로 |
| <code>@GetMapping</code>     | GET                          | 조회        |
| <code>@PostMapping</code>    | POST                         | 생성        |
| <code>@PutMapping</code>     | PUT                          | 전체 수정     |
| <code>@PatchMapping</code>   | PATCH                        | 부분 수정     |
| <code>@DeleteMapping</code>  | DELETE                       | 삭제        |

- 불일치 시 오류: `produces` 불일치는 406, `consumes` 불일치는 415

# 핸들러 파라미터가 값을 꺼내는 위치

---

| 어노테이션/타입        | 값 위치             | 언제              |
|-----------------|------------------|-----------------|
| @PathVariable   | 경로 템플릿 변수        | 리소스 식별          |
| @RequestParam   | 쿼리 스트링, 폼        | 검색어·페이지 번호      |
| @RequestBody    | 요청 본문 전체         | JSON을 DTO로 역직렬화 |
| @ModelAttribute | 여러 요청 파라미터       | 파라미터를 필드에 바인딩   |
| @RequestHeader  | 요청 헤더            | 인증 토큰 등         |
| @CookieValue    | 쿠키               | 세션 식별자 등        |
| @RequestPart    | 멀티파트 파트          | 파일 업로드          |
| Pageable        | page, size, sort | 페이징·정렬 조건       |



# @RequestBody와 @ModelAttribute의 차이

---

- @RequestBody: 본문 하나를 Jackson이 통째로 역직렬화
- @ModelAttribute: 쿼리·폼의 개별 파라미터를 필드에 하나씩 바인딩
- 혼용 불가: JSON 본문에 @ModelAttribute를 쓰면 바인딩되지 않음

# 선택적 쿼리 파라미터 받기

---

```
1  @GetMapping("/boards")
2  PageResponse<BoardSummaryResponse> list(
3      @RequestParam(defaultValue = "0") int page,
4      @RequestParam(required = false) String keyword) {
5      return boardService.getList(page, keyword);
6  }
```

# DTO와 응답 제어

# 엔티티를 그대로 주고받지 않는 이유

---

- 내부 구현 노출: 웹 계층이 영속 계층의 필드 구조에 직접 결합됨
- 순환 참조·지연 로딩 프록시 직렬화: 연관 엔티티를 직렬화하다 예외 또는 무한 루프 발생
- 의도치 않은 필드 노출: 비밀번호 해시 같은 민감 필드가 응답에 그대로 실림
- record로 계약 고정: 불변 값 타입으로 요청·응답 형태를 명시적으로 못 박음

# 클라이언트가 보내는 값만 담은 요청 DTO

---

```
1 record BoardCreateRequest(  
2     @NotBlank @Size(max = 100) String title,  
3     @NotBlank String content) {  
4 }
```

# 엔티티를 응답 형태로 옮기는 팩터리

---

```
1 record BoardResponse(Long id, String title, String content,  
2                       String authorNickname, Instant createdAt) {  
3     static BoardResponse from(Board board) {  
4         return new BoardResponse(board.getId(), board.getTitle(),  
5                                   board.getContent(), board.getAuthor().getNickname(),  
6                                   board.getCreatedAt());  
7     }  
8 }
```

# 요청 본문에 검증 걸기

---

```
1  @PostMapping
2  ResponseEntity<BoardResponse> create(
3      @Valid @RequestBody BoardCreateRequest request) {
4      ...
5  }
```

# REST에서 BindingResult를 두지 않는 이유

---

- SSR의 관례: `BindingResult`로 검증 실패를 잡아 같은 폼 뷰로 되돌아감
- 돌아갈 화면 없음: REST 응답은 화면이 아니므로 로컬에서 흡수할 이유가 없음
- 예외 그대로 전파: 선언하지 않으면 `MethodArgumentNotValidException`이 던져짐
- 일관 변환은 Advice에서: `@RestControllerAdvice`가 오류 응답으로 변환(형식 변환은 다음 차시)



# 응답을 조립하는 빌더 순서

---

```
1 return ResponseEntity
2     .status(HttpStatus.CREATED)           // 1. 상태 코드
3     .location(location)                   // 2. 표준 헤더
4     .header("X-Request-Id", requestId)   // 3. 사용자 정의 헤더
5     .body(response);                     // 4. 본문
```

# ResponseEntity 팩터리 메서드 (1/2)

---

| 시작 메서드                    | 상태             | 반환 빌더                       | 주요 용도                                   |
|---------------------------|----------------|-----------------------------|-----------------------------------------|
| <code>ok()</code>         | 200 OK         | <code>BodyBuilder</code>    | 조회 및 수정 성공 시 데이터와 함께 반환                 |
| <code>created(uri)</code> | 201 Created    | <code>BodyBuilder</code>    | 리소스 생성 성공 및 <code>Location</code> 헤더 전달 |
| <code>accepted()</code>   | 202 Accepted   | <code>BodyBuilder</code>    | 비동기 작업 요청 접수 완료                         |
| <code>noContent()</code>  | 204 No Content | <code>HeadersBuilder</code> | 본문 없이 성공 상태만 전달                         |

- `HeadersBuilder` 특징: `body()` 메서드가 생략되어 204 응답에 실수로 본문을 포함하는 오류를 차단함

# ResponseEntity 팩터리 메서드 (2/2)

---

| 시작 메서드                      | 상태              | 반환 빌더                       | 주요 용도                            |
|-----------------------------|-----------------|-----------------------------|----------------------------------|
| <code>badRequest()</code>   | 400 Bad Request | <code>BodyBuilder</code>    | 요청 파라미터 또는 본문 검증 실패 처리           |
| <code>notFound()</code>     | 404 Not Found   | <code>HeadersBuilder</code> | 요청한 리소스가 존재하지 않음을 반환             |
| <code>status(status)</code> | 동적 지정           | <code>BodyBuilder</code>    | 표준 메서드가 없는 상태 코드(예: 409, 422) 지정 |

- `HeadersBuilder` 특징: `notFound()` 또한 본문 없이 404 헤더 응답을 명확히 조립함
- `status( ... )` 활용: `HttpStatus` Enum 또는 숫자 코드를 전달하여 상황별 응답을 자유롭게 구사함

# 생성 응답에 Location 헤더 담기

---

```
1  @PostMapping
2  ResponseEntity<BoardResponse> create(@Valid @RequestBody BoardCreateRequest request) {
3      BoardResponse response = boardService.create(request.title(), request.content());
4      URI location = ServletUriComponentsBuilder.fromCurrentRequest()
5          .path("/{id}").buildAndExpand(response.id()).toUri();
6      return ResponseEntity.created(location).body(response);
7  }
```

# 본문 없는 응답을 표현하는 Void

---

- `void` 키워드: 자바 키워드라 제네릭 타입 인자로 쓸 수 없음
- `Void` 참조 타입: 값이 없음을 제네릭에서 표현하며 인스턴스를 만들 수 없음
- 계약 표현: `ResponseEntity<Void>` 는 상태·헤더만 반환하고 본문은 없다는 뜻
- 자동 204 아님: 반환 타입이 `void` 라도 상태 코드가 자동으로 204가 되지는 않음
- 삭제 응답: `ResponseEntity.noContent().build()` 로 본문 없이 204를 명시

# ResponseEntity와 DTO 직접 반환의 선택 기준

---

| 상황                            | 선택                                       | 이유            |
|-------------------------------|------------------------------------------|---------------|
| 상태 고정, 헤더 불필요                 | DTO 직접 반환 + <code>@ResponseStatus</code> | 시그니처가 간결      |
| 상태가 조건에 따라 달라짐                | <code>ResponseEntity</code>              | 분기하며 동적 결정    |
| <code>Location</code> 등 헤더 필요 | <code>ResponseEntity</code>              | 본문과 헤더를 함께 구성 |
| 본문 없는 성공(204)                 | <code>noContent().build()</code>         | 본문 타입 강제 없음   |

# 게시판 API 구현과 문서화

# 게시판 컨트롤러의 뼈대

---

```
1  @RestController
2  @RequestMapping("/api/boards")
3  @RequiredArgsConstructor
4  class BoardApiController {
5      private final BoardService boardService;
6  }
```



# 목록과 상세를 돌려주는 조회 API

---

```
1  @GetMapping
2  PageResponse<BoardSummaryResponse> list(Pageable pageable) {
3      return boardService.getList(pageable);
4  }
5
6  @GetMapping("/{id}")
7  BoardResponse detail(@PathVariable("id") Long id) {
8      return boardService.getDetail(id);
9  }
```

# 리소스를 통째로 교체하는 수정 API

---

```
1  @PutMapping("/{id}")
2  BoardResponse update(@PathVariable("id") Long id,
3                      @Valid @RequestBody BoardUpdateRequest request) {
4      return boardService.update(id, request.title(), request.content());
5  }
```

# 본문 없이 끝나는 삭제 API

---

```
1 @DeleteMapping("/{id}")
2 ResponseEntity<Void> delete(@PathVariable("id") Long id) {
3     boardService.delete(id);
4     return ResponseEntity.noContent().build();
5 }
```

# 수정·삭제 응답을 정하는 기준

---

- 수정 응답 선택: 변경된 리소스 본문과 함께 `200 OK` 를 반환하거나, 본문 없이 `204 No Content` 반환 중 팀 컨벤션으로 통일
- 삭제 응답 관례: 삭제 성공 시 본문 없이 `204 No Content` 반환이 표준 관례
- 시간 표현 규격: 생성·수정 시각은 `Instant` 타입으로 통일하고 UTC ISO 8601 형식(`2026-08-12T14:30:00Z`)으로 반환
- 동시성 보호 필요성: 여러 사용자가 동시에 동등한 대상을 수정할 때 덮어쓰기(Lost Update)를 방지하는 검증 계약 필요

# 동시 수정 제어: ETag와 If-Match

---

## ETag (Entity Tag)

리소스의 특정 상태나 버전을 식별하기 위해 서버가 응답 헤더로 전달하는 고유 해시/버전 값

- **ETag (응답 헤더):** GET 조회 시 서버가 해당 리소스의 현재 버전 식별자를 전달 ( ETag: "v1.0.2" )
- **If-Match (요청 헤더):** PUT / PATCH 수정 요청 시 클라이언트가 조회했던 기존 버전 식별자를 첨부
- **충돌 검증 흐름:**
  - 버전 일치: 수정 승인 후 200 OK (새 ETag 발행)
  - 버전 불일치: 다른 사용자가 먼저 수정했으므로 412 Precondition Failed 로 덮어쓰기 차단

# Page를 그대로 반환하지 않는 이유

---

- 내부 구현 종속: `Page<T>`를 그대로 반환하면 JSON 구조가 `PageImpl` 내부 구현에 묶임
- 공식 경고: Spring Data도 `Page` 직접 노출을 권장하지 않는다고 명시
- 계약 고정: 클라이언트가 실제로 쓰는 필드만 골라 응답 계약으로 못 박음

# 페이징 응답을 감싸는 record

---

```
1 record PageResponse<T>(List<T> content, int page, int size,  
2                          long totalElements, int totalPages) {  
3     static <T> PageResponse<T> from(Page<T> page) {  
4         return new PageResponse<>(page.getContent(), page.getNumber(),  
5                                   page.getSize(), page.getTotalElements(), page.getTotalPages());  
6     }  
7 }
```

# 명세는 자동 생성 후 부족한 곳만 보강

---

- 자동 생성: 애노테이션 없이도 컨트롤러·DTO 분석만으로 기본 명세가 만들어짐
- 먼저 확인: 구현 후 Swagger UI에서 추론된 결과를 먼저 훑어봄
- 추론 한계: 예외 응답이나 보안 정책 같은 조건부 계약은 자동으로 알 수 없음
- 과잉 주석 경계: 기계적으로 모든 필드를 반복 표기하면 구현과 문서가 함께 복잡해짐



# OpenAPI 문서화 어노테이션

---

| 어노테이션        | 위치         | 역할               |
|--------------|------------|------------------|
| @Tag         | 컨트롤러 클래스   | API 그룹명·설명       |
| @Operation   | 핸들러 메서드    | 요약·상세 설명         |
| @ApiResponse | 핸들러 메서드    | 상태 코드별 응답 스키마    |
| @Parameter   | 파라미터       | 설명과 예시값          |
| @Schema      | DTO 필드·클래스 | 필드 설명, 예시, 필수 여부 |
| @Hidden      | 클래스·메서드    | 문서 대상 제외         |

# 필요한 곳만 설명을 붙인 조회 API

---

```
1  @Tag(name = "게시판", description = "게시글 CRUD API")
2  @Operation(summary = "게시글 상세 조회")
3  @ApiResponse(responseCode = "200", description = "조회 성공")
4  @GetMapping("/{id}")
5  BoardResponse detail(
6      @Parameter(description = "게시글 id", example = "1")
7      @PathVariable("id") Long id) {
8      return boardService.getDetail(id);
9  }
```

# 학습 요약 (1/2)

---

- 설계 축: 리소스는 URI로, 동작은 HTTP 메서드로, 결과는 상태 코드로 표현
- 무상태: 각 요청이 처리에 필요한 문맥을 스스로 포함하도록 설계
- URI 규칙: 복수 명사·소문자 경로, 조회 조건은 쿼리 스트링으로 분리
- 응답 계약: 201은 Location 헤더, 204는 본문 없음을 지켜 예측 가능성 확보

# 학습 요약 (2/2)

---

- 컨트롤러: `@RestController`의 반환값은 메시지 컨버터가 JSON으로 직렬화
- DTO 분리: 엔티티 대신 `record` 요청·응답 DTO로 API 계약을 고정
- 응답 제어: 상태·헤더가 필요하다면 `ResponseEntity`, 고정이면 DTO 직접 반환
- 문서화: Swagger UI를 먼저 열어두고 자동 명세에서 부족한 부분만 보강